

QUIZ-1 QUESTION PAPER
EC2.204 Introduction to Processor Architectures

Total Marks:	60
Duration:	90 minutes
Number of Questions:	4

RISC-V ASSEMBLY LANGUAGE PROGRAMMING [40 marks]

Instructions:

- Answer ALL questions in the space provided and use the last page for rough work.
- For multiple choice questions, clearly write your option (A, B, C, or D) in the Answer field.
- Show all working for descriptive questions.
- Assume RV32I base instruction set unless otherwise specified.
- Register conventions: a0-a7 for arguments, t0-t6 for temporaries, s0-s11 for saved registers.

Question 1: Multiple Choice Questions [12 marks]

Answer all six MCQs. Each question has exactly one correct answer [2 marks each].

Reference: Basic GEMM C Code

```
// C = A * B, where A is MxK, B is KxN, C is MxN
void gemm(int M, int N, int K, int *A, int *B, int *C) {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            int sum = 0;
            for (int k = 0; k < K; k++) {
                sum += A[i*K + k] * B[k*N + j];
            }
            C[i*N + j] = sum;
        }
    }
}
```

1.1 In the GEMM code above, what is the total number of multiply-accumulate operations for matrices where $M=N=K=n$?

- (A) n^2
- (B) n^3
- (C) $2n^3$
- (D) $n^2 \log n$

Answer: _____

1.2 Which RISC-V instruction is used to compute the product $A[i*K + k] * B[k*N + j]$?

- (A) add
- (B) mul
- (C) mla
- (D) madd

Answer: _____

1.3 Consider the following RISC-V assembly snippet:

```
slli t1, t0, 2
add t2, a0, t1
lw t3, 0(t2)
```

What does this code accomplish if a0 contains the base address of an integer array and t0 contains an index?

- (A) Stores t0 to memory
- (B) Loads the element at index t0 from the array into t3
- (C) Computes $t0 * 4$ and stores in t3
- (D) Loads the element at index $t0/4$ from the array

Answer: _____

1.4 In RISC-V, the instruction 'blt t0, t1, loop' performs:

- (A) Branch to loop if $t0 > t1$ (signed)
- (B) Branch to loop if $t0 < t1$ (signed)
- (C) Branch to loop if $t0 == t1$
- (D) Branch to loop if $t0 \geq t1$ (unsigned)

Answer: _____

1.5 For the GEMM inner loop, which memory access pattern exhibits the WORST cache locality?

- (A) Accessing $A[i * K + k]$ with k incrementing by 1
- (B) Accessing $B[k * N + j]$ with k incrementing by 1
- (C) Accessing $C[i * N + j]$ with j incrementing by 1
- (D) All have equal cache locality

Answer: _____

1.6 What is the purpose of the 'jalr ra, ra, 0' instruction?

- (A) Jump to address in ra and save return address in ra
- (B) Return from a function call
- (C) Call a function at address 0
- (D) Clear the return address register

Answer: _____

Question 2: C to RISC-V Assembly Translation [18 marks]

Consider the following simplified GEMM kernel that computes a single element of the output matrix:

```
// Compute C[i][j] = sum of A[i][k] * B[k][j] for k = 0 to K-1
// Assume row-major storage, all matrices have dimension N x N
int dot_product(int *A, int *B, int i, int j, int N, int K) {
    int sum = 0;
    for (int k = 0; k < K; k++) {
        sum += A[i*N + k] * B[k*N + j];
    }
    return sum;
}
```

Register Allocation:

- a0: Base address of matrix A
- a1: Base address of matrix B
- a2: Value of i
- a3: Value of j
- a4: Value of N
- a5: Value of K
- Return value in a0

(a) Complete the following RISC-V assembly implementation by filling in the **six blanks** (labeled [BLANK_1] through [BLANK_6]) in the table provided. [12 marks]
[2 marks each]

```
dot_product:
    # Prologue (if needed)
    li    t0, 0          # t0 = k (loop counter)
    li    t1, 0          # t1 = sum (accumulator)

    # Compute row offset for A: i * N * 4
    mul   t2, a2, a4      # t2 = i * N (element count)
    [BLANK_1]             # t2 = i * N * 4 (convert element count to byte offset, x4)
    add   t2, t2, t0      # t2 = &A[i*N] (base for row i)

    # Compute column offset for B: j * 4
    slli  t3, a3, 2       # t3 = j * 4

loop:
    # Check loop condition: k < K
    [BLANK_2]             # if k >= K, exit loop (exit label is "done")

    # Load A[i*N + k]
    slli  t4, t0, 2       # t4 = k * 4
    add   t4, t2, t4      # t4 = &A[i*N + k]
    [BLANK_3]             # t5 = A[i*N + k]

    # Load B[k*N + j]
    mul   t6, t0, a4      # t6 = k * N
    slli  t6, t6, 2       # t6 = k * N * 4
    add   t6, a1, t6      # t6 = &B[k*N]
    add   t6, t6, t3      # t6 = &B[k*N + j]
    lw    s0, 0(t6)       # s0 = B[k*N + j]

    # Multiply and accumulate
    [BLANK_4]             # s1 = A[i*N+k] * B[k*N+j]
    [BLANK_5]             # sum += product
```

```

# Increment k
addi t0, t0, 1
[BLANK_6]          # jump back to loop

done:
mv    a0, t1       # return sum
ret

```

Blank	Answer	Purpose
[BLANK_1]		
[BLANK_2]		
[BLANK_3]		
[BLANK_4]		
[BLANK_5]		
[BLANK_6]		

(b) The code above uses register s0. According to RISC-V calling conventions, what additional code is required in the prologue and epilogue? Write the necessary instructions. [6 marks]
[2 marks each]

Prologue (allocate stack space).

Epilogue (deallocate stack space).

Question 3: Assembly Code Correctness [10 marks]

The following RISC-V assembly code attempts to implement a 2×2 matrix multiplication $C = A \times B$ where all matrices are stored in row-major order. The code contains **THREE errors (commented)**.

```
# Matrix multiplication: C = A * B (2x2 matrices)
# a0 = address of A, a1 = address of B, a2 = address of C
# All matrices are 2x2, stored in row-major order
```

```
matmul 2x2:
    # C[0][0] = A[0][0]*B[0][0] + A[0][1]*B[1][0]
    lw    t0, 0(a0)          # t0 = A[0][0]
    lw    t1, 0(a1)          # t1 = B[0][0]
    mul    t2, t0, t1         # t2 = A[0][0] * B[0][0]
    lw    t0, 4(a0)          # t0 = A[0][1]
    lw    t1, 4(a1)          # ERROR 1
    mul    t3, t0, t1
    add    t2, t2, t3
    sw    t2, 0(a2)          # C[0][0] = t2

    # C[0][1] = A[0][0]*B[0][1] + A[0][1]*B[1][1]
    lw    t0, 0(a0)          # t0 = A[0][0]
    lw    t1, 4(a1)          # t1 = B[0][1]
    mul    t2, t0, t1
    lw    t0, 4(a0)          # t0 = A[0][1]
    lw    t1, 12(a1)         # t1 = B[1][1]
    mul    t3, t0, t1
    add    t2, t2, t3
    sw    t2, 4(a2)          # C[0][1] = t2

    # C[1][0] = A[1][0]*B[0][0] + A[1][1]*B[1][0]
    lw    t0, 8(a0)          # t0 = A[1][0]
    lw    t1, 0(a1)          # t1 = B[0][0]
    mul    t2, t0, t1
    lw    t0, 12(a0)         # t0 = A[1][1]
    lw    t1, 8(a1)          # t1 = B[1][0]
    mul    t3, t0, t1         # ERROR 2
    add    t2, t2, t3
    sw    t2, 8(a2)          # C[1][0] = t2

    # C[1][1] = A[1][0]*B[0][1] + A[1][1]*B[1][1]
    lw    t0, 8(a0)          # t0 = A[1][0]
    lw    t1, 4(a1)          # t1 = B[0][1]
    mul    t2, t0, t1
    lw    t0, 12(a0)         # t0 = A[1][1]
    lw    t1, 12(a1)         # t1 = B[1][1]
    mul    t3, t0, t1
    add    t2, t2, t3
    sw    t2, 16(a2)         # ERROR 3

    ret
```

(a) For each error, state: [6 marks]
[2 marks each]

- What the error is (Explanation)
- The corrected instruction (Assembly Code)

C = A * B
(commented).

Error	Incorrect Code	Correct Assembly Code	Explanation
1			
2			
3			

(b) Given the following input matrices, trace through the corrected code and compute the output matrix C: [2 marks]

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

(c) The memory layout for matrix A (in the question (b) above) starting at address 0x1000 is:

Address	0x1000	0x1004	0x1008	0x100C
Value	xxxxxx	xxxxxx	xxxxxx	xxxxxx

What value is stored at each address? [2 marks]

Address	0x1000	0x1004	0x1008	0x100C
Value				

CPU Performance Metrics – Quick Reference

Performance Equations

$$\text{CPU Time} = \text{IC} \times \text{CPI} \times T = (\text{IC} \times \text{CPI}) / f$$

$$\text{Performance} = 1 / \text{CPU Time}$$

$$\text{Speedup} = \text{Time}_{\text{old}} / \text{Time}_{\text{new}} = \text{Perf}_{\text{new}} / \text{Perf}_{\text{old}}$$

CPI Calculation

$$\text{CPI} = \Sigma (\text{CPI}_i \times \text{Frequency}_i) \text{ for all instruction types } i$$

$$\text{Clock Cycles} = \text{IC} \times \text{CPI}$$

Amdahl's Law

$$\text{Speedup}_{\text{overall}} = 1 / [(1 - f) + (f / S)]$$

$$\text{Maximum Speedup} = 1 / (1 - f) \quad [\text{when } S \rightarrow \infty]$$

$$\text{For parallel processing with } n \text{ processors: } S = n$$

Power and Energy

$$\text{Dynamic Power} = C \times V^2 \times f$$

$$\text{Static Power} \approx \text{constant (leakage current)}$$

$$\text{Total Power} = \text{Dynamic Power} + \text{Static Power}$$

$$\text{Energy} = \text{Power} \times \text{Time}$$

$$\text{Performance per Watt} = 1 / (\text{Time} \times \text{Power})$$

Common Unit Conversions

Prefix	Value	Example
Giga (G)	10^9	3 GHz = 3×10^9 Hz
Mega (M)	10^6	500 MHz = 5×10^8 Hz
Nano (n)	10^{-9}	2 ns = 2×10^{-9} s
Pico (p)	10^{-12}	500 pF = 5×10^{-10} F

Question 4: Comprehensive Design Trade-off (20 marks)

You are designing a mobile processor and have three design options:

Design	Clock Rate	CPI	Voltage	Capacitance
Design A	2.0 GHz	1.2	1.0 V	2.0 nF
Design B	2.5 GHz	1.5	1.1 V	2.2 nF
Design C	1.5 GHz	0.9	0.9 V	1.8 nF

For a program with 10 billion instructions:

(a) Calculate the execution time for each design. [3 marks]

(b) Calculate the dynamic power for each design. [3 marks]

(c) Calculate the energy consumed by each design. [3 marks]

(d) Calculate the performance per watt for each design. [6 marks]

(e) Which design would you recommend and why? Consider both performance and energy constraints.
[5 marks]